Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Department of Information Technology

ASSIGNMENT 6

NAME	Viraj Shrikant Pathare
PRN	124B1F149
CLASS	SY IT
DIVISION	A

1. Schema Overview

Five core entities from the Employee Appraisal System ER diagram. Audit tables are used by triggers.

```
CREATE TABLE Employee (emp_id INT PK, name VARCHAR(100), email VARCHAR(150) UNIQUE,
phone VARCHAR(15), dept_id INT FK, designation VARCHAR(80),
joining_date DATE, salary DECIMAL(10,2), is_active TINYINT DEFAULT 1);

CREATE TABLE Department (dept_id INT PK, dept_name VARCHAR(100));

CREATE TABLE Appraisal (appraisal_id INT PK, emp_id INT FK, reviewer_id INT FK,
appraisal_date DATE, rating DECIMAL(3,1), comments TEXT,
status ENUM('Pending','Completed','Approved') DEFAULT 'Pending');

CREATE TABLE Goals (goal_id INT PK, emp_id INT FK, description VARCHAR(255),
target_date DATE, achieved TINYINT DEFAULT 0);

CREATE TABLE Salary_History (history_id INT PK AUTO_INCREMENT, emp_id INT FK,
old_salary DECIMAL(10,2), new_salary DECIMAL(10,2),
effective_date DATE, reason VARCHAR(200));

-- Audit tables used by triggers
CREATE TABLE Audit_Log (log_id INT PK AUTO_INCREMENT, table_name VARCHAR(50),
action VARCHAR(20), record_id INT, old_value TEXT,
new_value TEXT, change_time DATETIME DEFAULT NOW());

CREATE TABLE Deleted_Employees_Log (log_id INT PK AUTO_INCREMENT,
emp_id INT, name VARCHAR(100), email VARCHAR(150), deleted_at DATETIME);
```

2. BEFORE INSERT Triggers

Fire before a new row is written. Used to validate, sanitise, or auto-populate values.

2.1 Validate & Normalise New Employee

Normalises email to lowercase, rejects negative salary, validates 10-digit phone.

```
DELIMITER $$
CREATE TRIGGER trg_before_insert_employee
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
SET NEW.email = LOWER(TRIM(NEW.email));

IF NEW.salary <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Salary must be positive.';
END IF;
```

```
IF NEW.phone IS NOT NULL AND NEW.phone NOT REGEXP '^[0-9]{10}$' THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Phone must be exactly 10 digits.';
END IF;

IF NEW.joining_date IS NULL THEN
SET NEW.joining_date = CURDATE();
END IF;
END $$
DELIMITER ;
```

2.2 Validate Appraisal Rating Range

Ensures rating is between 1.0 and 5.0; auto-sets appraisal_date if missing.

```
DELIMITER $$
CREATE TRIGGER trg_before_insert_appraisal
BEFORE INSERT ON Appraisal
FOR EACH ROW
BEGIN
IF NEW.rating < 1.0 OR NEW.rating > 5.0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Rating must be between 1.0 and 5.0.';
END IF;

IF NEW.emp_id = NEW.reviewer_id THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Employee cannot review themselves.';
END IF;

IF NEW.appraisal_date IS NULL THEN
SET NEW.appraisal_date = CURDATE();
END IF;
END $$
DELIMITER ;
```

2.3 Validate Goal Target Date

Rejects goal insertion if target date is in the past.

```
DELIMITER $$
CREATE TRIGGER trg_before_insert_goal
BEFORE INSERT ON Goals
FOR EACH ROW
BEGIN
IF NEW.target_date < CURDATE() THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Goal target date cannot be in the past.';
END IF;

IF NEW.achieved IS NULL THEN
SET NEW.achieved = 0;
END IF;
END $$
DELIMITER ;
```

3. AFTER INSERT Triggers

Fire after the new row is committed. Ideal for audit logs and derived-table updates.

3.1 Log New Employee in Audit Table

```
DELIMITER $$
CREATE TRIGGER trg_after_insert_employee
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
INSERT INTO Audit_Log (table_name, action, record_id, new_value)
VALUES ('EMPLOYEE', 'INSERT', NEW.emp_id,
```

```
CONCAT('Name:', NEW.name, ' Email:', NEW.email,
' Salary:', NEW.salary));
END $$
DELIMITER ;
```

3.2 Auto-Create Salary History on New Employee

Inserts an initial salary history record when a new employee is registered.

```
DELIMITER $$
CREATE TRIGGER trg_after_insert_salary_init
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
INSERT INTO Salary_History (emp_id, old_salary, new_salary,
effective_date, reason)
VALUES (NEW.emp_id, 0, NEW.salary, NEW.joining_date, 'Initial Salary');
END $$
DELIMITER ;
```

3.3 Log New Appraisal Submission

```
DELIMITER $$
CREATE TRIGGER trg_after_insert_appraisal
AFTER INSERT ON Appraisal
FOR EACH ROW
BEGIN
INSERT INTO Audit_Log (table_name, action, record_id, new_value)
VALUES ('APPRAISAL', 'INSERT', NEW.appraisal_id,
CONCAT('EmpID:', NEW.emp_id, ' Rating:', NEW.rating,
' Status:', NEW.status));
END $$
DELIMITER ;
```

4. BEFORE UPDATE Triggers

Intercept row changes before persistence — validate new values and block illegal transitions.

4.1 Prevent Illegal Appraisal Status Transitions

An Approved appraisal cannot revert to Pending; Pending cannot jump directly to Approved.

```
DELIMITER $$
CREATE TRIGGER trg_before_update_appraisal
BEFORE UPDATE ON Appraisal
FOR EACH ROW
BEGIN
IF OLD.status = 'Approved' AND NEW.status = 'Pending' THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Approved appraisal cannot revert to Pending.';
END IF;

IF OLD.status = 'Pending' AND NEW.status = 'Approved' THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Pending appraisal must be Completed first.';
END IF;
END $$
DELIMITER ;
```

4.2 Salary Change Validation

Prevents salary reduction below 80% of existing salary and rejects negative values.

```
DELIMITER $$
CREATE TRIGGER trg_before_update_salary
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
IF NEW.salary <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Salary must be positive.';
END IF;

IF NEW.salary < (OLD.salary * 0.80) THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Salary cannot be reduced by more than 20%.';
END IF;
END $$
DELIMITER ;
```

4.3 Email Normalisation on Update

```
DELIMITER $$
CREATE TRIGGER trg_before_update_employee_email
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
IF NEW.email <> OLD.email THEN
SET NEW.email = LOWER(TRIM(NEW.email));
END IF;
END $$
DELIMITER ;
```

5. AFTER UPDATE Triggers

React to committed changes and propagate effects to other tables or logs.

5.1 Log Salary Change in Salary_History

```
DELIMITER $$
CREATE TRIGGER trg_after_update_salary
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
IF OLD.salary <> NEW.salary THEN
INSERT INTO Salary_History (emp_id, old_salary, new_salary,
effective_date, reason)
```

```
VALUES (NEW.emp_id, OLD.salary, NEW.salary, CURDATE(),
'Salary Update via Trigger');

INSERT INTO Audit_Log (table_name, action, record_id,
old_value, new_value)
VALUES ('EMPLOYEE', 'SALARY_CHANGE', NEW.emp_id,
CONCAT('Salary:', OLD.salary),
CONCAT('Salary:', NEW.salary));
END IF;
END $$
DELIMITER ;
```

5.2 Log Appraisal Status Change

```
DELIMITER $$
CREATE TRIGGER trg_after_update_appraisal_status
AFTER UPDATE ON Appraisal
FOR EACH ROW
BEGIN
IF OLD.status <> NEW.status THEN
INSERT INTO Audit_Log (table_name, action, record_id,
old_value, new_value)
VALUES ('APPRAISAL', 'STATUS_CHANGE', NEW.appraisal_id,
CONCAT('Status:', OLD.status),
CONCAT('Status:', NEW.status));
END IF;
END $$
DELIMITER ;
```

5.3 Mark All Goals Pending When Appraisal Reverts to Pending

```
DELIMITER $$
CREATE TRIGGER trg_after_update_appraisal_reset_goals
AFTER UPDATE ON Appraisal
FOR EACH ROW
BEGIN
IF OLD.status <> 'Pending' AND NEW.status = 'Pending' THEN
UPDATE Goals SET achieved = 0
WHERE emp_id = NEW.emp_id;
END IF;
END $$
DELIMITER ;
```

6. BEFORE DELETE Triggers

Prevent deletion of records that have dependent active data.

6.1 Block Deletion of Employee With Active Appraisal

```
DELIMITER $$
CREATE TRIGGER trg_before_delete_employee
BEFORE DELETE ON Employee
FOR EACH ROW
BEGIN
DECLARE v_active INT;
SELECT COUNT(*) INTO v_active
FROM Appraisal
WHERE emp_id = OLD.emp_id AND status IN ('Pending','Completed');

IF v_active > 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Cannot delete employee with active appraisal.';
END IF;
END $$
DELIMITER ;
```

6.2 Block Deletion of Department With Active Employees

```
DELIMITER $$
CREATE TRIGGER trg_before_delete_department
BEFORE DELETE ON Department
FOR EACH ROW
BEGIN
DECLARE v_count INT;
SELECT COUNT(*) INTO v_count FROM Employee
WHERE dept_id = OLD.dept_id AND is_active = 1;

IF v_count > 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Cannot delete department with active employees.';
END IF;
END $$
DELIMITER ;
```

7. AFTER DELETE Triggers

7.1 Archive Deleted Employee Details

```
DELIMITER $$
CREATE TRIGGER trg_after_delete_employee
AFTER DELETE ON Employee
FOR EACH ROW
BEGIN
INSERT INTO Deleted_Employees_Log (emp_id, name, email, deleted_at)
VALUES (OLD.emp_id, OLD.name, OLD.email, NOW());

INSERT INTO Audit_Log (table_name, action, record_id, old_value)
VALUES ('EMPLOYEE', 'DELETE', OLD.emp_id,
CONCAT('Name:', OLD.name, ' Email:', OLD.email));
END $$
DELIMITER ;
```

7.2 Clean Up Goals and Training When Employee Is Deleted

```
DELIMITER $$
CREATE TRIGGER trg_after_delete_employee_cleanup
AFTER DELETE ON Employee
FOR EACH ROW
BEGIN
DELETE FROM Goals WHERE emp_id = OLD.emp_id;
DELETE FROM Training WHERE emp_id = OLD.emp_id;

INSERT INTO Audit_Log (table_name, action, record_id, old_value)
VALUES ('EMPLOYEE', 'CLEANUP', OLD.emp_id,
```

```
CONCAT('Goals and Training removed for emp_id:', OLD.emp_id));
END $$
DELIMITER ;
```

8. INSTEAD OF Trigger (View-Based)

Replaces the default DML action on a VIEW, routing updates to base tables. MySQL uses a BEFORE trigger on the base table as an equivalent.

8.1 Create the Employee-Appraisal Summary View

```
CREATE VIEW emp_appraisal_summary AS
SELECT e.emp_id, e.name, e.email,
a.appraisal_id, a.status AS appr_status,
a.rating, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
JOIN Appraisal a ON e.emp_id = a.emp_id;
```

8.2 MySQL BEFORE UPDATE Equivalent (routes view updates to base tables)

```
DELIMITER $$
CREATE TRIGGER trg_before_update_via_view
BEFORE UPDATE ON Appraisal
FOR EACH ROW
BEGIN
IF NEW.status NOT IN ('Pending','Completed','Approved') THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Invalid appraisal status via view.';
END IF;
END $$
DELIMITER ;
```

9. Compound / Statement-Level Trigger Simulation (MySQL)

MySQL simulates compound trigger behaviour using session variables to bridge BEFORE and AFTER triggers.

```
-- Simulation: compound trigger on Appraisal INSERT
DELIMITER $$
CREATE TRIGGER trg_before_insert_appr_compound
BEFORE INSERT ON Appraisal
FOR EACH ROW
BEGIN
IF NEW.rating < 1.0 OR NEW.rating > 5.0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Rating must be between 1.0 and 5.0.';
END IF;
SET @last_appr_action = 'INSERT';
END $$

CREATE TRIGGER trg_after_insert_appr_compound
AFTER INSERT ON Appraisal
FOR EACH ROW
BEGIN
INSERT INTO Audit_Log (table_name, action, record_id, new_value)
VALUES ('APPRAISAL', @last_appr_action, NEW.appraisal_id,
CONCAT('Rating:', NEW.rating, ' Status:', NEW.status));
END $$
DELIMITER ;
```